Lecture 14: Matrix decompositions, NMF

Jamie Haddock

Table of contents

1 Eigenvalue decomposition **1**
1.1 Eigenvalues and eigenvectors . . . 1
1.2 Eigenvalue decomposition . . . 1
1.3 Conditioning of eigenvalues . . . 2
1.4 Computing the EVD . . . 5

2 Singular value decomposition **6**
2.1 SVD . . . 7
2.2 Connections to the EVD . . . 7
2.3 Thin form . . . 7
2.4 SVD and the 2-norm . . . 8

3 Nonnegative matrix factorization **9**
3.1 Nonnegative matrix factorization . . . 9

1 Eigenvalue decomposition

Other than solving linear systems (or linear least-squares), the most important problem in numerical linear algebra is the eigenvalue problem.

1.1 Eigenvalues and eigenvectors

Exercise: Do you recall the definition of eigenvalue and eigenvector?

Suppose $\mathbf{A}$ is a matrix, and λ and $\mathbf{x}$ are an eigenvalue and eigenvector, respectively. What do you know about $\mathbf{A}$, λ, and $\mathbf{x}$? What is the relationship between them?

Answer:

Given a square matrix $\mathbf{A}$, if

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$$

for a scalar λ and a nonzero vector $\mathbf{x}$, then λ is an **eigenvalue** and $\mathbf{x}$ is an associated **eigenvector**.

1.2 Eigenvalue decomposition

We can rewrite the eigenvalue definition as $(\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = 0$. Thus $(\mathbf{A} - \lambda\mathbf{I})$ is singular, and must have a zero determinant.

The determinant $\det(\mathbf{A} - \lambda\mathbf{I})$ is called the **characteristic polynomial**. Its roots are the eigenvalues, so we know that an $n \times n$ matrix has n eigenvalues, counting algebraic multiplicity.

Suppose that $\mathbf{A}\mathbf{v}_k = \lambda_k \mathbf{v}_k$ for $k = 1, \dots, n$. We can summarize these as

$$\begin{bmatrix} \mathbf{A}\mathbf{v}_1 & \mathbf{A}\mathbf{v}_2 & \cdots & \mathbf{A}\mathbf{v}_n \end{bmatrix} = \begin{bmatrix} \lambda_1\mathbf{v}_1 & \lambda_2\mathbf{v}_2 & \cdots & \lambda_n\mathbf{v}_n \end{bmatrix},$$

$$\mathbf{A}\begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \end{bmatrix} = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \end{bmatrix} \begin{bmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n \end{bmatrix},$$

which we write as

$$\mathbf{AV} = \mathbf{VD}.$$

If $\mathbf{V}$ is a nonsingular matrix, then we arrive at the eigenvalue decomposition.

Definition: Eigenvalue decomposition (EVD)

An **eigenvalue decomposition** (EVD) of a square matrix $\mathbf{A}$ is

$$\mathbf{A} = \mathbf{VDV}^{-1}.$$

If $\mathbf{A}$ has an EVD, we say that $\mathbf{A}$ is **diagonalizable**; otherwise $\mathbf{A}$ is **nondiagonalizable** (or *defective*).

One simple example of a nondiagonalizable matrix is

$$\mathbf{B} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

There are instances in which we can guarantee an EVD exists.

Theorem:

If the $n \times n$ matrix $\mathbf{A}$ has n distinct eigenvalues, then $\mathbf{A}$ is diagonalizable.

1.3 Conditioning of eigenvalues

Just as linear systems have condition numbers that quantify the effect of finite precision, eigenvalue problems may be poorly conditioned too. Many possible results exist, we will use just one.

Theorem: Bauer–Fike

Let $\mathbf{A} \in \mathbb{C}^{n\times n}$ be diagonalizable, $\mathbf{A} = \mathbf{VDV}^{-1}$, with eigenvalues $\lambda_1, \dots, \lambda_n$. If μ is an eigenvalue of $\mathbf{A} + \mathbf{E}$ for a complex matrix $\mathbf{E}$, then

$$\min_{j=1,\dots,n} |\mu - \lambda_j| \le \kappa(\mathbf{V})\, \|\mathbf{E}\|,$$

where $\|\cdot\|$ and κ are in the 2-norm.

The Bauer–Fike theorem tells us that eigenvalues can be perturbed by an amount that is $\kappa(\mathbf{V})$ times larger than perturbations to the matrix.

This result is a bit less straightforward than it might seem—eigenvectors are not unique, so there are multiple possible values for $\kappa(\mathbf{V})$. Even so, the theorem indicates caution when a matrix has eigenvectors that form an ill-conditioned matrix.

The limiting case of $\kappa(\mathbf{V}) = \infty$ might be interpreted as indicating a nondiagonalizable matrix $\mathbf{A}$. The other extreme is also of interest: $\kappa(\mathbf{V}) = 1$, which implies that $\mathbf{V}$ is unitary.

> Definition: Normal matrix
>
> If $\mathbf{A}$ has an EVD with a unitary eigenvector matrix $\mathbf{V}$, then $\mathbf{A}$ is a **normal matrix**.

In Math 73, we refer to unitary matrices as **orthogonal** and normal matrices as **orthogonally diagonalizable matrices**.

Hermitian and real symmetric matrices are normal. Since the condition number of a unitary matrix is equal to 1, Bauer-Fike guarantees that a perturbation of a normal matrix changes the eigenvalues by the same amount or less.

```
  Project No packages added to or removed from `~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
 Manifest No packages added to or removed from `~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
  Project No packages added to or removed from `~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
 Manifest No packages added to or removed from `~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
  Project No packages added to or removed from `~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
 Manifest No packages added to or removed from `~/Documents/GitHub/Math-189M-NumericalLinearAlgebra/
```

```
%%julia
n = 7
A = randn(n,n) + 1im*randn(n,n)
A = (A+A')/2    # define a Hermitian matrix

 ,V = eigen(A)
@show cond(V);
```

```
cond(V) = 1.0000000000000007
```

```
%%julia
ΔA = 1e-8*normalize(randn(n,n) + 1im*randn(n,n))
̃ = eigvals(A+ΔA)     # perturb the matrix and calculate the perturbed eigenvalues
dist = minimum( [abs(x-y) for x in ̃, y in  ], dims=2 ) # measure the effect on evs
```

```
7×1 Matrix{Float64}:
 1.214564821570518e-9
 2.843593267320583e-10
 1.2040567543573146e-9
 1.3124727156346435e-9
 1.3049292540607081e-9
 1.4592888021599655e-9
 1.0273055024667163e-9
```

As promised, the perturbation of the eigenvalues does not exceed the norm of the perturbation to the matrix.

Now, we try a triangular matrix.

```
%%julia
n = 20
x = 1:n
A = triu( x*ones(n)' )
A[1:5,1:5]
```

```
5×5 Matrix{Float64}:
 1.0  1.0  1.0  1.0  1.0
 0.0  2.0  2.0  2.0  2.0
 0.0  0.0  3.0  3.0  3.0
 0.0  0.0  0.0  4.0  4.0
 0.0  0.0  0.0  0.0  5.0
```

```
%%julia
 ,V = eigen(A)
@show cond(V);  # far from normal
```

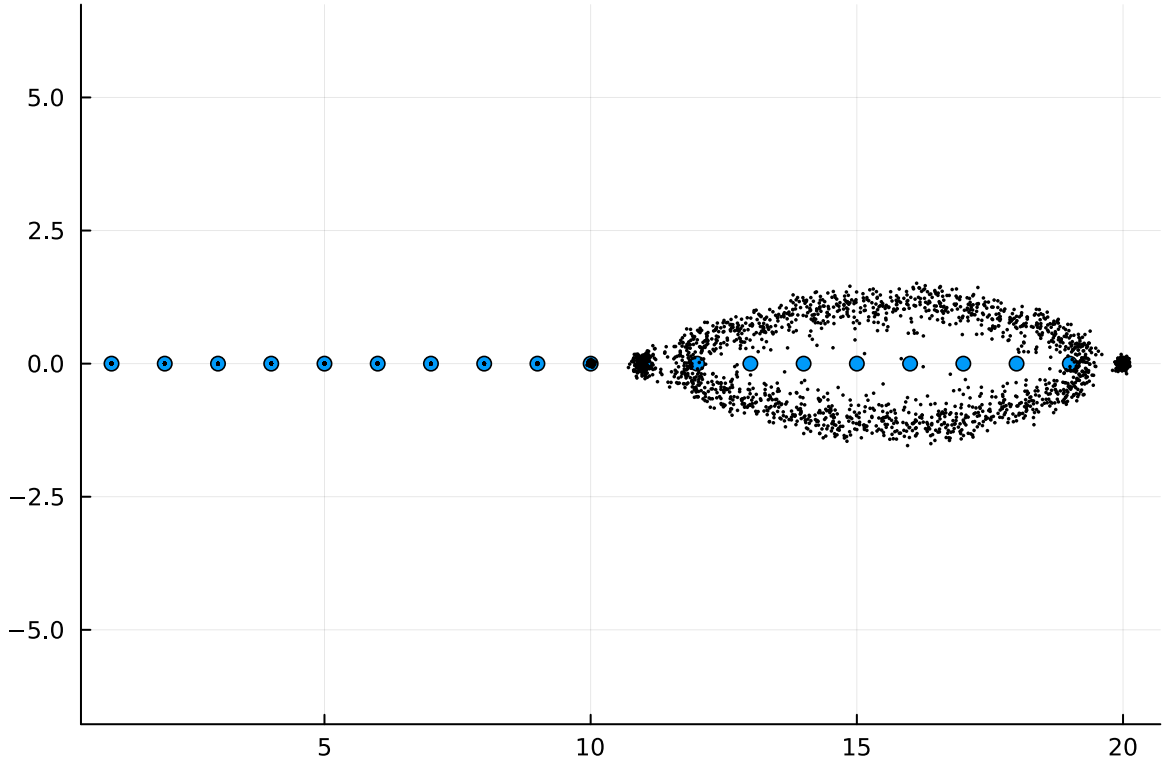
```
cond(V) = 6.1499066649293165e9
```

```
%%julia
ΔA = 1e-8*normalize(randn(n,n) + 1im*randn(n,n))
˜ = eigvals(A+ΔA)
dist = minimum( [abs(x-y) for x in ˜, y in  ], dims=2 )
BF_bound = cond(V)*norm(ΔA)      # eigenvalues can change by a lot more
@show maximum(dist),BF_bound;
```

```
(maximum(dist), BF_bound) = (0.4722237188988462, 61.49906664929318)
```

If we plot the eigenvalues of many perturbations, we get a cloud of points that roughly represents all the possible eigenvalues when representing this matrix with single-precision accuracy.

```
%%julia
plt = scatter( ,zeros(n),aspect_ratio=1, legend=false)
for _ in 1:200
    ΔA = eps(Float32)*normalize(randn(n,n) + 1im*randn(n,n))
    ˜ = eigvals(A+ΔA)
    scatter!(real(˜),imag(˜),m=1,color=:black)
end
plt
```



Some eigenvalues are far more affected by the perturbation than others!

1.4 Computing the EVD

In practice, root-finding on the characteristic polynomial is not used in numerical methods for eigenvalue computation. In fact, numerical eigenvalue computation methods are often used to compute the roots of given polynomials.

Many numerical methods use matrix powers, and build on the fact that the EVD simplifies matrix powers significantly:

$$\mathbf{A}^2 = \mathbf{V}\mathbf{D}\mathbf{V}^{-1}\mathbf{V}\mathbf{D}\mathbf{V}^{-1} = \mathbf{V}\mathbf{D}^2\mathbf{V}^{-1}$$

and more generally,

$$\mathbf{A}^k = \mathbf{V}\mathbf{D}^k\mathbf{V}^{-1}.$$

If the eigenvalues have different magnitudes, then as $k \to \infty$ the entries on the diagonal of $\mathbf{D}^k$ will become increasingly well separated and easy to pick out.

```
%%julia
D = diagm( [-6,-1,2,4,5] )
V,R = qr(randn(5,5))    # V is unitary
A = V*D*V'
```

```
5×5 Matrix{Float64}:
 -1.73511   -3.7173    1.28833   -0.159862  -2.77977
```

```
 -3.7173     0.447082  1.2988    -1.35875   -0.478328
  1.28833    1.2988    1.27405    2.08744    0.643382
 -0.159862  -1.35875   2.08744    2.00742    1.67068
 -2.77977   -0.478328  0.643382   1.67068    2.00655
```

```
%%julia
eigvals(A)
```

```
5-element Vector{Float64}:
 -5.999999999999995
 -1.0000000000000002
  2.0000000000000004
  3.999999999999997
  4.999999999999999
```

```
%%julia
Q,R = qr(A)
A = R*Q;
```

```
%%julia
eigvals(A)  # this is a similarity transformation, so the eigenvalues are unchanged
```

```
5-element Vector{Float64}:
 -5.999999999999998
 -0.9999999999999998
  2.0000000000000004
  3.9999999999999973
  4.9999999999999964
```

Interestingly, if we repeat this transformation many times, the resulting matrix converges to $\mathbf{D}$:

```
%%julia
for k in 1:40
    Q,R = qr(A)
    A = R*Q
end
A
```

```
5×5 Matrix{Float64}:
 -6.0           0.00191116    5.64167e-7    1.01014e-16  -3.27679e-16
  0.00191116    5.0           0.00054119   -2.88028e-16  -8.66066e-16
  5.64167e-7    0.00054119    4.0          -3.89098e-14  -3.62505e-16
 -2.3625e-19   -3.22014e-16  -3.98302e-14   2.0           1.3471e-12
 -9.85527e-32  -3.37393e-28   5.83916e-25   1.3472e-12   -1.0
```

The process demonstrated here is known as the *Francis QR iteration*, and it can be formulated as an $O(n^3)$ algorithm for finding the EVD. Such an algorithm is the foundation of what the `eigen` function uses.

2 Singular value decomposition

The SVD is as fundamental as the EVD and very related to it!

2.1 SVD

Definition: Singular value decomposition (SVD)

The **singular value decomposition** of an $m \times n$ matrix $\mathbf{A}$ is

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^*,$$

where $\mathbf{U} \in \mathbb{C}^{m\times m}$ and $\mathbf{V} \in \mathbb{C}^{n\times n}$ are unitary and $\mathbf{S} \in \mathbb{R}^{m\times n}$ is real and diagonal with nonnegative elements.
The columns of $\mathbf{U}$ and $\mathbf{V}$ are called **left** and **right singular vectors**, respectively. The diagonal elements of $\mathbf{S}$, written $\sigma_1, \dots, \sigma_r$, for $r = \min\{m, n\}$, are called the **singular values** of $\mathbf{A}$ and are ordered so that

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r \geq 0, \qquad r = \min\{m, n\}.$$

We call σ_1 the **principal singular value** and $\mathbf{u}_1$ and $\mathbf{v}_1$ the **principal singular vectors**.

Theorem:

Every $m \times n$ matrix has an SVD. The singular values of a matrix are unique, but the singular vectors are not. If the matrix is real, then $\mathbf{U}$ and $\mathbf{V}$ in {eq}`svd` can be chosen to be real, orthogonal matrices.

2.2 Connections to the EVD

Theorem:

The nonzero eigenvalues of $\mathbf{A}^*\mathbf{A}$ are the squares of the singular values of $\mathbf{A}$.

Exercise: Prove this!

Another way to write $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^*$ is

$$\mathbf{A}\mathbf{V} = \mathbf{U}\mathbf{S}.$$

Taken columnwise, this equation means

$$\mathbf{A}\mathbf{v}_k = \sigma_k \mathbf{u}_k, \qquad k = 1, \dots, r = \min\{m, n\}.$$

In words, each right singular vector is mapped by $\mathbf{A}$ to a scaled version of its corresponding left singular vector; the magnitude of scaling is its singular value.

This is like the EVD! The SVD sacrifices having the same basis in both source and image spaces (they may not even have the same dimension).

2.3 Thin form

We saw earlier in this course that a matrix has both *full* and *thin* forms of the QR factorization. A similar situation holds with the SVD.

Suppose $\mathbf{A}$ is $m \times n$ with $m > n$ and let $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^*$ be an SVD. The last $m - n$ rows of $\mathbf{S}$ are all zero due to the fact that $\mathbf{S}$ is diagonal. Hence

$$\mathbf{US} = \begin{bmatrix} \mathbf{u}_1 & \cdots & \mathbf{u}_n & \mathbf{u}_{n+1} & \cdots & \mathbf{u}_m \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_n \\ & 0 & \end{bmatrix} = \begin{bmatrix} \mathbf{u}_1 & \cdots & \mathbf{u}_n \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_n \end{bmatrix}$$

$$= \hat{\mathbf{U}}\hat{\mathbf{S}},$$

in which $\hat{\mathbf{U}}$ is $m \times n$ and $\hat{\mathbf{S}}$ is $n \times n$.

This allows us to define the **thin SVD**

$$\mathbf{A} = \hat{\mathbf{U}}\hat{\mathbf{S}}\mathbf{V}^*,$$

in which $\hat{\mathbf{S}}$ is square and diagonal and $\hat{\mathbf{U}}$ is ONC but not square.

2.4 SVD and the 2-norm

The SVD is intimately connected to the 2-norm, as the following theorem describes.

Theorem:

Let $\mathbf{A} \in \mathbb{C}^{m \times n}$ have an SVD $\mathbf{A} = \mathbf{USV}^*$. Then:

1. The 2-norm satisfies
$$\|\mathbf{A}\|_2 = \sigma_1.$$
2. The rank of $\mathbf{A}$ is the number of nonzero singular values.
3. Let $r = \min\{m, n\}$. Then
$$\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2 \|\mathbf{A}^+\|_2 = \frac{\sigma_1}{\sigma_r},$$
where a division by zero implies that $\mathbf{A}$ does not have full rank.

```
%%julia
A = [i^j for i=1:5, j=0:3]
```

```
5×4 Matrix{Int64}:
 1  1   1    1
 1  2   4    8
 1  3   9   27
 1  4  16   64
 1  5  25  125
```

```
%%julia
 = svdvals(A)
```

```
4-element Vector{Float64}:
 146.69715365883005
   5.738569780953702
   0.9998486640841026
```

```
0.11928082685241923
```

```
%%julia
@show opnorm(A,2);
@show  [1];
```

```
opnorm(A, 2) = 146.69715365883005
 [1] = 146.69715365883005
```

```
%%julia
@show cond(A,2);
@show  [1]/ [end];
```

```
cond(A, 2) = 1229.846887633767
 [1] /  [var"end"] = 1229.846887633767
```

```
%%julia
U, ,V = svd(A);    # thin form is default
@show size(U);
@show size(V);
```

```
size(U) = (5, 4)
size(V) = (4, 4)
```

```
%%julia
@show opnorm(U'*U - I);   # verify orthogonality
@show opnorm(V'*V - I);
```

```
opnorm(U' * U - I) = 5.35490754941372e-16
opnorm(V' * V - I) = 7.483935019298407e-16
```

3 Nonnegative matrix factorization

While the EVD and SVD are very fundamental matrix decompositions and have very clear linear algebraic interpretations, they are not the only decompositions of interest. One very popular decomposition is the **nonnegative matrix factorization** (NMF) – the nonnegativity of the factors makes it very interpretable.

3.1 Nonnegative matrix factorization

> Definition: Nonnegative matrix factorization (NMF)
>
> Given $\mathbf{A} \in \mathbb{R}^{m \times n}$, the **nonnegative matrix factorization** (NMF) of $\mathbf{A}$ is a decomposition of the form
>
> $$\mathbf{A} \approx \mathbf{WH},$$
>
> where $\mathbf{W} \in \mathbb{R}^{m \times r}$ and $\mathbf{H} \in \mathbb{R}^{r \times n}$ are nonnegative matrices, and r is the user-selected rank of the factorization.

Note that this factorization is not exact – one typically attempts to find $\mathbf{W}$ and $\mathbf{H}$ by minimizing some error between the data $\mathbf{A}$ and the approximation $\mathbf{WH}$.

```
%%julia
# Load MNIST data
train_x, _ = MNIST(split=:train)[:]
```

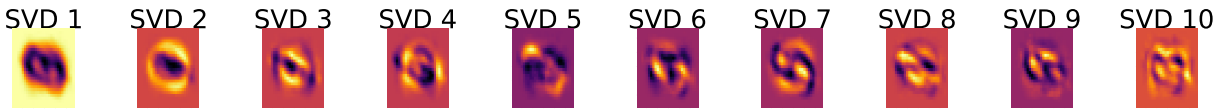
```
X = Float64.(reshape(train_x[:, :, 1:500], :, 500))  # Use first 500 digits

# Normalize pixel values
X ./= 255.0;
```

```
%%julia
U, S, Vt = svd(X);
```

```
%%julia
# Visualize first 10 SVD components (U columns)
function show_components(U, title_prefix)
    n = 10
    plt = plot(layout = (1, n), size = (100 * n, 100))
    for i in 1:n
        img = reshape(U[:, i], 28, 28)
        heatmap!(plt[i], img, axis = false, colorbar = false, title = "$title_prefix $i")
    end
    display(plt)
end

show_components(U, "SVD")
```




```
%%julia
k = 10
model = nnmf(X, k; maxiter = 500)
W = model.W
H = model.H;
```

```
%%julia
show_components(W, "NMF")
```

